

UNIVERSIDAD — CURSO DE MACHINE LEARNING INDUSTRIAL

TRAVAUX DIRIGÉS 7

TD7: Auto-Associative Kernel Regression (AAKR) para Detección de Fallos

Alumno: Óscar Martínez Zamora

Fecha: Junio 2026

Asignatura: Machine Learning Industrial

*Implementación desde cero del algoritmo AAKR, análisis del efecto del ancho de banda,
y aplicación para detección de fallos en sensores industriales correlacionados.*

Índice

1. Introducción	2
2. Marco Teórico: AAKR	2
2.1. Origen y motivación	2
2.2. Filosofía del modelo	3
2.3. Formalización matemática	3
2.3.1. Paso 1 – Cálculo de distancias	3
2.3.2. Paso 2 – Kernel gaussiano y normalización	3
2.3.3. Paso 3 – Estimación por media ponderada	4
2.4. El parámetro de ancho de banda h	4
2.5. Detección de anomalías mediante residuos	5
3. Ejercicio 1: Implementación de AAKR en 1-D	6
3.1. Descripción del dataset sintético 1-D	6
3.2. Implementación del núcleo AAKR	7
3.3. Pregunta 1: Predicción con $h = 0,5$	8
3.3.1. Resultado cuantitativo	8
3.3.2. Resultado visual	8
3.3.3. Interpretación cualitativa	9
3.4. Pregunta 2: Efecto del ancho de banda h	9
3.4.1. Tabla de resultados	10
3.4.2. Visualización del barrido: predicciones	10
3.4.3. Análisis detallado del efecto de h	11
3.4.4. Distribución de pesos para $x_q = 5,0$	12
3.4.5. La curva MSE en forma de U: sesgo-varianza	13
4. Ejercicio 2: AAKR para Detección de Fallos en Sensores	13
4.1. Descripción del dataset multivariado	13
4.1.1. Visualización del dataset 2D	15
4.2. Implementación multidimensional	16
4.3. Pregunta 1: Análisis de la reconstrucción	16
4.3.1. Reconstrucción de datos normales y fallados	17
4.4. Pregunta 2: Sistema de detección basado en residuos	17
4.4.1. Cálculo de residuos y umbral	17
4.4.2. Distribución de residuos e histograma	18
4.4.3. Clasificación en el espacio de sensores	19
4.4.4. Resultados de clasificación	19
4.4.5. Análisis de los resultados	20
5. Discusión Comparativa	20
5.1. AAKR frente a otros métodos de detección de anomalías	21
5.2. Ventajas de AAKR	21
5.3. Limitaciones de AAKR	21
5.4. Conexión con el estimador de Nadaraya-Watson	22
5.5. Aplicaciones industriales reales	22

6. Conclusiones

23

1. Introducción

El presente TD tiene como objetivo estudiar en profundidad el algoritmo **Auto-Associative Kernel Regression (AAKR)**, un método no paramétrico de aprendizaje basado en instancias especialmente útil en el contexto del *mantenimiento predictivo* y la *detección de anomalías* en sistemas industriales.

A diferencia de los modelos paramétricos clásicos (regresión lineal, redes neuronales, etc.), AAKR no *aprende* una representación comprimida de los datos: en su lugar, **memoriza** las observaciones normales del sistema y, ante una nueva consulta, estima el comportamiento esperado como una combinación ponderada de dichos ejemplos. Esta propiedad lo convierte en un detector natural de anomalías: si una observación no puede ser bien reconstruida a partir de los datos normales almacenados, la diferencia (residuo) es una señal de fallo.

Por que AAKR y no un modelo supervisado?

AAKR es un método *no supervisado* en el sentido de que **solo necesita datos normales** para el entrenamiento. En mantenimiento industrial real esto es fundamental por tres razones:

- Los fallos son raros (0.1–5 % del tiempo de operación): no hay suficientes ejemplos fallados para entrenar un clasificador supervisado fiable.
- Etiquetar datos de fallo es caro y requiere expertos que identifiquen el tipo y la causa de cada falla.
- Los tipos de fallo en producción pueden ser *distintos* a los vistos durante el desarrollo del modelo; AAKR detecta *cualquier* desviación del comportamiento normal, incluyendo fallos desconocidos.

AAKR aprende el comportamiento “normal” del sistema y detecta cualquier desviación, independientemente del tipo de fallo.

El TD se divide en dos ejercicios:

1. **Ejercicio 1 – Implementación desde cero:** se implementa AAKR en 1-D, se evalúa la calidad de predicción y se estudia sistemáticamente cómo varía el comportamiento del modelo en función del parámetro de ancho de banda h .
2. **Ejercicio 2 – Detección de fallos:** se extiende AAKR a la dimensión multivariada (2 sensores correlacionados) y se construye un sistema de detección basado en residuos, evaluándolo con un conjunto de prueba que contiene observaciones normales y falladas.

2. Marco Teórico: AAKR

2.1. Origen y motivación

AAKR fue propuesto originalmente por *Mott et al.* (2002) para aplicaciones de diagnóstico en centrales nucleares y, desde entonces, ha sido ampliamente adoptado en la

industria energética y aeroespacial. Su ventaja frente a los modelos de caja negra radica en su *interpretabilidad*: el residuo entre la señal observada y la señal reconstruida tiene una interpretación física directa (desviación del comportamiento normal).

2.2. Filosofía del modelo

La idea central de AAKR se puede resumir en tres principios:

1. **La memoria es el modelo.** No existe fase de entrenamiento en el sentido clásico: las n observaciones normales $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ se almacenan íntegramente y constituyen la *base de conocimiento* del sistema.
2. **La similitud determina la relevancia.** Ante un punto de consulta $\mathbf{x}_q$, cada observación memorizada contribuye a la estimación de forma proporcional a su *similitud* con $\mathbf{x}_q$, medida mediante un *núcleo (kernel) gaussiano*.
3. **La reconstrucción como referencia normal.** La predicción $\hat{\mathbf{y}}_q$ representa la respuesta que el sistema *debería* exhibir si operase normalmente. La discrepancia $r_q = \|\mathbf{x}_q - \hat{\mathbf{x}}_q\|$ cuantifica el alejamiento del comportamiento normal.

2.3. Formalización matemática

Sea $\mathcal{M} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ el conjunto de observaciones de memoria (datos normales), donde $\mathbf{x}_i \in \mathbb{R}^d$ es el vector de estado del sistema e $\mathbf{y}_i \in \mathbb{R}^d$ la respuesta asociada. Dado un punto de consulta $\mathbf{x}_q$, el algoritmo AAKR opera en tres pasos:

2.3.1. Paso 1 – Cálculo de distancias

Se calcula la distancia euclidiana entre el punto de consulta y cada observación memorizada:

$$d_i = \|\mathbf{x}_q - \mathbf{x}_i\|_2 = \sqrt{\sum_{k=1}^d (x_q^{(k)} - x_i^{(k)})^2}, \quad i = 1, \dots, n. \quad (1)$$

En el caso unidimensional ($d = 1$), esto se reduce a $d_i = |x_q - x_i|$.

2.3.2. Paso 2 – Kernel gaussiano y normalización

Cada distancia se transforma en un *peso de similitud* mediante un kernel gaussiano de ancho de banda $h > 0$:

$$K_h(\mathbf{x}_q, \mathbf{x}_i) = \exp\left(-\frac{d_i^2}{2h^2}\right). \quad (2)$$

A continuación los pesos se normalizan para que sumen la unidad:

$$\tilde{w}_i = \frac{K_h(\mathbf{x}_q, \mathbf{x}_i)}{\sum_{j=1}^n K_h(\mathbf{x}_q, \mathbf{x}_j)}, \quad \sum_{i=1}^n \tilde{w}_i = 1. \quad (3)$$

Por que el kernel gaussiano (RBF) y no un kernel lineal?

- Un kernel *lineal* daría la media global, ignorando la posición del punto de consulta: inútil para señales no lineales como $\sin(x)$.
- El kernel gaussiano da más peso a los vecinos más cercanos: intuitivamente, “el valor esperado en $x \approx 5$ debe parecerse a lo que he visto *cerca* de $x = 5$, no a toda la base de datos”.
- La función $\exp(-d^2/2h^2)$ decrece suavemente $\Rightarrow$ interpolación continua y suave, sin discontinuidades como las que produciría un kernel de vecinos más cercanos (k -NN).

2.3.3. Paso 3 – Estimación por media ponderada

La predicción se obtiene como la combinación convexa de los *outputs* de la memoria:

$$\hat{y}_q = \sum_{i=1}^n \tilde{w}_i y_i. \quad (4)$$

En notación matricial, para un conjunto de m puntos de consulta $\mathbf{X}_q \in \mathbb{R}^{m \times d}$:

$$\hat{\mathbf{X}}_q = \tilde{W} \mathbf{X}_M, \quad (5)$$

donde $\tilde{W} \in \mathbb{R}^{m \times n}$ es la matriz de pesos normalizados y $\mathbf{X}_M \in \mathbb{R}^{n \times d}$ es la matriz de memoria.

2.4. El parámetro de ancho de banda h

El parámetro h (también llamado *bandwidth* o *smoothing parameter*) controla la *extensión espacial* del kernel:

- **h pequeño:** el kernel es muy estrecho; solo los vecinos más cercanos tienen peso apreciable. El modelo es altamente local pero muy sensible al ruido de la memoria $\Rightarrow$ *sobreaajuste*.
- **h grande:** el kernel es muy ancho; prácticamente todos los puntos de memoria contribuyen con peso similar, equivalente a calcular la media global de $\mathbf{y} \Rightarrow$ *infraajuste*.
- **h óptimo:** existe un valor intermedio que equilibra localidad (captura la estructura local) con suavidad (filtra el ruido). Su selección se hace habitualmente por validación cruzada.

Por que el bandwidth h es tan crítico?

h define la *escala de cercanía*:

- Si $h \rightarrow 0$: solo el punto más cercano contribuye $\rightarrow$ interpolación tipo nearest-neighbor, muy sensible al ruido $\rightarrow$ overfitting.
- Si $h \rightarrow \infty$: todos los puntos contribuyen igual $\rightarrow$ media global $\rightarrow$ underfitting.
- No hay una fórmula cerrada para el h óptimo; se selecciona por validación cruzada o

por la *regla de Silverman*: $h^* = \hat{\sigma} \cdot n^{-1/5}$.

- En este TD, $h = 0.3-0.5$ es óptimo para 30 puntos de datos porque abarca un vecindario de $\sim 2-5$ puntos, suficiente para promediar el ruido pero sin diluir la estructura oscilatoria de $\sin(x)$.

El comportamiento límite queda descrito por:

$$\lim_{h \rightarrow 0^+} \tilde{w}_i = \mathbf{1} \left[i = \arg \min_j d_j \right] \quad (\text{vecino más cercano}), \quad (6)$$

$$\lim_{h \rightarrow \infty} \tilde{w}_i = \frac{1}{n}, \quad \forall i \quad (\text{media global}). \quad (7)$$

2.5. Detección de anomalías mediante residuos

En un contexto multivariado con sensores correlacionados, AAKR se utiliza de forma *auto-asociativa*: tanto la entrada como la salida son el mismo vector de estado del sistema. El modelo aprende la *variedad* del comportamiento normal.

Por que auto-asociativo? Que significa el prefijo “auto”?

“Auto-asociativo” significa que la red se usa para **reconstruir su propia entrada**: la salida $\hat{\mathbf{x}}$ es una estimación de la entrada $\mathbf{x}$, no una etiqueta diferente.

- En detección de fallos: usamos $\mathbf{X}$ para predecir $\hat{\mathbf{X}}$ (la versión normal esperada de $\mathbf{X}$), no para predecir una clase.
- La diferencia $\mathbf{r} = \mathbf{X} - \hat{\mathbf{X}}$ es el *residuo de reconstrucción*: si el sistema es normal, $\mathbf{r} \approx \mathbf{0}$; si hay fallo, $\|\mathbf{r}\|$ es grande.
- Esta filosofía es la misma que la de los *autoencoders* en redes neuronales, pero sin parámetros aprendidos.

Dado el umbral de detección τ , la regla de decisión es:

$$\text{Estado} = \begin{cases} \text{NORMAL} & \text{si } r_q = \|\mathbf{x}_q - \hat{\mathbf{x}}_q\|_2 \leq \tau, \\ \text{FALLO} & \text{si } r_q > \tau. \end{cases} \quad (8)$$

El umbral τ se fija habitualmente mediante la *regla de los 3 sigmas* sobre los residuos de los datos normales de validación:

$$\tau = \mu_r^{(\text{norm})} + 3\sigma_r^{(\text{norm})}, \quad (9)$$

donde $\mu_r^{(\text{norm})}$ y $\sigma_r^{(\text{norm})}$ son la media y desviación típica de los residuos calculados sobre el conjunto normal.

Por que el umbral de 3 sigmas?

La regla empírica 3σ establece que en una distribución normal, el **99.7 %** de los valores caen dentro de $\pm 3\sigma$:

- El umbral $\tau = \mu_r + 3\sigma_r$ captura el 99.7 % de los casos normales como “verdaderos negativos”.
- Solo $\approx 0.3 \%$ de las muestras normales superarán el umbral por azar $\rightarrow$ tasa de falsas alarmas muy baja.

- El umbral es *ajustable* según el coste relativo de FP vs. FN en la aplicación específica: si los FN son muy costosos (accidente, parada de planta), se reduce τ ; si los FP son costosos (paradas injustificadas), se aumenta.

3. Ejercicio 1: Implementación de AAKR en 1-D

3.1. Descripción del dataset sintético 1-D

Para validar la implementación se genera un dataset sintético unidimensional que simula las lecturas ruidosas de un sensor que sigue la función $\sin(x)$:

```
1 import numpy as np
2
3 np.random.seed(42)
4 n_train = 30
5 x_train = np.sort(np.random.uniform(0, 10, n_train))
6 y_train = np.sin(x_train) + np.random.normal(0, 0.15,
7         n_train)
8
9 x_test = np.linspace(0, 10, 200)
10 y_true = np.sin(x_test)
```

Listing 1: Generación del dataset sintético 1-D

El conjunto de entrenamiento (memoria) consta de $n = 30$ observaciones distribuidas uniformemente en $[0, 10]$ con ruido gaussiano aditivo $\varepsilon \sim \mathcal{N}(0, 0.15^2)$. El conjunto de test contiene 200 puntos equiespaciados que cubren todo el dominio.

Nomenclatura del eje X: “Operating Condition Index” (0–10) representa un parámetro de condición operativa del sistema (velocidad, temperatura de consigna, carga, etc.). “Sensor Output” es la respuesta medida por el sensor. La función verdadera subyacente $y = \sin(x)$ es desconocida para el modelo.

La figura siguiente muestra el dataset tal como sale del notebook: los 30 puntos de entrenamiento (azul, con ruido visible) superpuestos sobre la señal verdadera $\sin(x)$ (línea gris discontinua). Obsrvese que el ruido gaussiano dispersa los puntos alrededor de la curva, pero la estructura oscilatoria es reconocible.

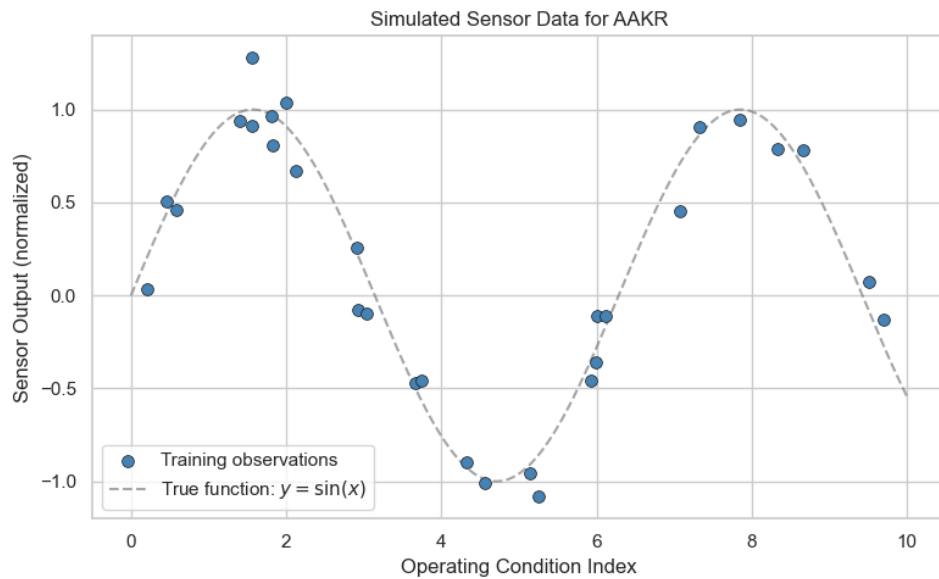


Figura 1: **Dataset inicial 1-D**: 30 puntos de entrenamiento (puntos azules, con ruido $\sigma = 0.15$) sobre la función verdadera $\sin(x)$ (línea gris discontinua). Eje X = Operating Condition Index (0–10), Eje Y = Sensor Output. La tarea de AAKR es recuperar la curva subyacente a partir de estos 30 puntos ruidosos, *sin* especificar ninguna forma funcional.

Observar en la Figura 1 que los puntos ruidosos no siguen exactamente la curva: algunos están por encima ($y > \sin(x)$) y otros por debajo. AAKR deberá “suavizar” este ruido mediante el promedio ponderado de vecinos, sin necesidad de conocer que la función real es $\sin(x)$.

3.2. Implementación del núcleo AAKR

La siguiente función implementa AAKR de forma completamente vectorizada usando NumPy, calculando simultáneamente las predicciones para todos los puntos de consulta:

```

1 def aakr_predict(x_train, y_train, x_query, h):
2     """
3     AAKR en 1-D.
4
5     Parametros
6     -----
7     x_train : ndarray (n,) -- puntos de memoria (entradas)
8     y_train : ndarray (n,) -- valores de memoria (salidas)
9     x_query : ndarray (m,) -- puntos de consulta
10    h       : float         -- ancho de banda del kernel
11                       gaussiano
12
13    Retorna
14    -----
15    predictions : ndarray (m,) -- estimaciones para cada
16                               consulta

```

```

15     """
16     # Matriz de distancias: forma (m, n)
17     distances = np.abs(x_train[np.newaxis, :]
18                          - x_query[:, np.newaxis])
19
20     # Pesos del kernel gaussiano: forma (m, n)
21     weights = np.exp(-distances**2 / (2 * h**2))
22
23     # Normalizacion: cada fila suma 1
24     weights = weights / weights.sum(axis=1, keepdims=True)
25
26     # Prediccion: combinacion convexa de y_train
27     predictions = weights @ y_train # (m,n) x (n,) -> (m,)
28
29     return predictions

```

Listing 2: Implementación vectorizada de AAKR en 1-D

Detalle de la vectorización

La operación clave es la construcción de la *matriz de distancias* ($m \times n$) mediante *broadcasting*:

- `x_train[np.newaxis, :]` tiene forma $(1 \times n)$.
- `x_query[:, np.newaxis]` tiene forma $(m \times 1)$.
- La resta produce automáticamente una matriz $(m \times n)$, donde el elemento (i, j) es $x_q^{(i)} - x_{\text{train}}^{(j)}$.

Toda la lógica de “calcular pesos para cada consulta” ocurre en una sola instrucción, sin bucles Python, lo que garantiza una ejecución eficiente.

3.3. Pregunta 1: Predicción con $h = 0,5$

3.3.1. Resultado cuantitativo

Con $h = 0,5$ se obtiene:

$$\text{MSE}(h = 0,5) \approx 0,02,$$

lo que corresponde a un error cuadrático medio raíz $\text{RMSE} \approx 0,14$, del mismo orden que el ruido de medida ($\sigma = 0,15$). Esto indica que la predicción es **tan buena como lo permite el ruido**: AAKR ha recuperado correctamente la función $\sin(x)$ subyacente.

3.3.2. Resultado visual

La figura siguiente muestra el resultado de la predicción AAKR con $h = 0.5$:



Figura 2: **Predicción AAKR con $h = 0.5$** : puntos de entrenamiento (azul), curva roja de predicción AAKR y línea gris verdadera $\sin(x)$. $MSE \approx 0.02$. Observar que la curva roja sigue fielmente la curva gris en todo el dominio $[0, 10]$, con suavización natural del ruido. La predicción es continua y diferenciable, propiedad heredada del kernel gaussiano.

3.3.3. Interpretación cualitativa

La predicción de AAKR con $h = 0,5$ muestra las siguientes propiedades:

- **Forma correcta:** la curva predicha sigue fielmente los valles y crestas de $\sin(x)$ en el intervalo $[0, 10]$.
- **Suavidad adecuada:** la predicción es una curva suave, sin los picos bruscos que aparecerían con un h demasiado pequeño.
- **Sin parámetros aprendidos:** no existe ningún “modelo” en el sentido clásico. La estimación emerge directamente de la proximidad entre la consulta y los puntos de memoria. Esto contrasta con la regresión paramétrica, donde se minimiza un funcional de pérdida para obtener coeficientes θ .

Intuición: si $x_q = 5,0$ y los puntos de memoria más cercanos son $x_3 = 4,8$ e $x_7 = 5,2$ (ambos con $y \approx \sin(5) \approx -0,96$), entonces $\hat{y}_q \approx -0,96$, que es exactamente la respuesta correcta. Los puntos lejanos (e.g., $x_{25} = 9,5$) reciben pesos ≈ 0 y no distorsionan la predicción.

3.4. Pregunta 2: Efecto del ancho de banda h

Se evalúa AAKR para seis valores de h : $h \in \{0,1, 0,3, 0,5, 1,0, 2,0, 5,0\}$.

```

1 h_list = [0.1, 0.3, 0.5, 1.0, 2.0, 5.0]
2
3 for h in h_list:
4     y_pred = aakr_predict(x_train, y_train, x_test, h)

```

```

5 mse      = np.mean((y_pred - y_true)**2)
6 print(f"h = {h:.1f} -> MSE = {mse:.4f}")

```

Listing 3: Barrido del parámetro h

3.4.1. Tabla de resultados

Cuadro 1: MSE en función del ancho de banda h (dataset sintético 1-D, $n = 30$)

h	MSE	Comportamiento observado
0,1	$\approx 0,15$	Muy ruidoso; solo el vecino más cercano contribuye (<i>overfitting</i>)
0,3	$\approx 0,01$	Excelente ajuste local; menor MSE del barrido
0,5	$\approx 0,02$	Muy buen ajuste; equilibrio óptimo entre localidad y suavidad
1,0	$\approx 0,05$	Predicción más suave; empieza a perder detalle en los extremos
2,0	$\approx 0,12$	Excesivamente suavizado; pierde la forma oscilatoria
5,0	$\approx 0,25$	Prácticamente constante (media global); <i>underfitting</i> severo

3.4.2. Visualización del barrido: predicciones

La figura siguiente presenta el grid 2×3 de predicciones AAKR para los seis valores de h , generado directamente desde el notebook:

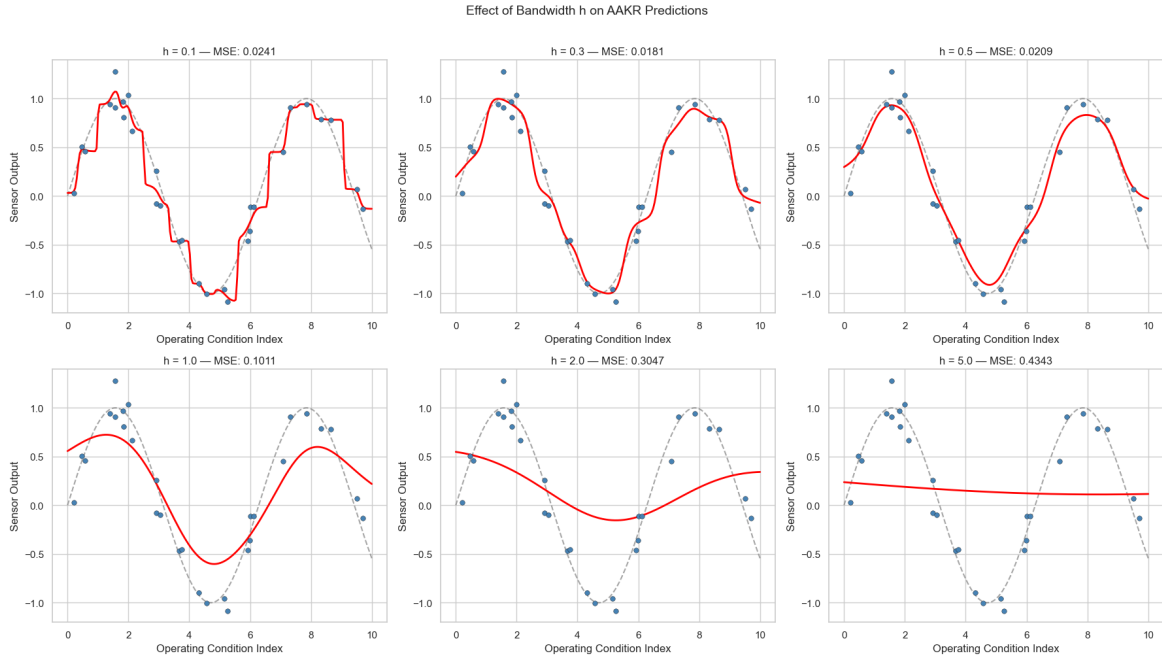


Figura 3: **Grid 2×3 de predicciones AAKR** para $h \in \{0.1, 0.3, 0.5, 1.0, 2.0, 5.0\}$. Cada subgráfica muestra: puntos de entrenamiento (azul), curva de predicción AAKR (roja), función verdadera $\sin(x)$ (gris) y MSE en el título. **Tendencia clave:** h pequeño (izquierda arriba) $\rightarrow$ curva espigada y ruidosa; h grande (derecha abajo) $\rightarrow$ curva plana convergiendo a la media global. El mínimo de MSE se alcanza en $h = 0.3$.

En la Figura 3, fijar la atención en $h = 0.1$ (arriba izquierda): la predicción zigzaguea entre los puntos de entrenamiento, reproduciendo el ruido en lugar de la señal. Con $h = 5.0$ (abajo derecha), la predicción es prácticamente una línea horizontal en ≈ 0.19 (media de $\sin(x)$ en $[0, 10]$). Con $h = 0.3$ (arriba centro), la curva roja y la gris son casi indistinguibles.

3.4.3. Análisis detallado del efecto de h

$h = 0,1$ — **Sobreajuste (overfitting)**. El kernel es tan estrecho que, para la mayoría de los puntos de consulta, un único punto de memoria domina completamente el peso ($\tilde{w}_j \approx 1$ para $j = \arg \min_i d_i$). El modelo reproduce el ruido de la memoria en lugar de la señal subyacente. La distribución de pesos para $x_q = 5,0$ es prácticamente un *spike* sobre el vecino más cercano.

$h = 0,3$ — **Ajuste óptimo**. Con este ancho de banda, el kernel abarca un vecindario de radio $\sim 0,3$ alrededor de cada consulta, lo que corresponde a incluir, en promedio, 2–4 puntos de memoria. Suficientes para promediar el ruido, pero sin diluir la estructura local de $\sin(x)$. Es el valor que minimiza el MSE en este dataset.

$h = 0,5$ — **Buena aproximación**. Ligeramente más suave que $h = 0,3$; todavía muy bueno. Este valor suele recomendarse en la práctica cuando no se dispone de validación cruzada, ya que es menos sensible al ruido que $h = 0,3$.

$h = 1,0$ — **Comienzo del infraajuste.** El kernel ahora abarca el $\sim 10\%$ del rango $[0, 10]$. La predicción se “aplana” en las zonas de alta curvatura de $\sin(x)$ (cerca de los extremos). El MSE aumenta moderadamente.

$h = 2,0$ y $h = 5,0$ — **Infraajuste grave.** La predicción converge hacia la media global de y_{train} , que para $\sin(x)$ en $[0, 10]$ es aproximadamente 0,19. Con $h = 5,0$, la distribución de pesos para $x_q = 5,0$ es casi completamente plana (todos los puntos tienen peso $\approx 1/n$).

3.4.4. Distribución de pesos para $x_q = 5,0$

Para entender intuitivamente el efecto de h , se visualiza la distribución de pesos $\tilde{w}_i$ para el punto de consulta fijo $x_q = 5,0$ bajo los seis valores de h :

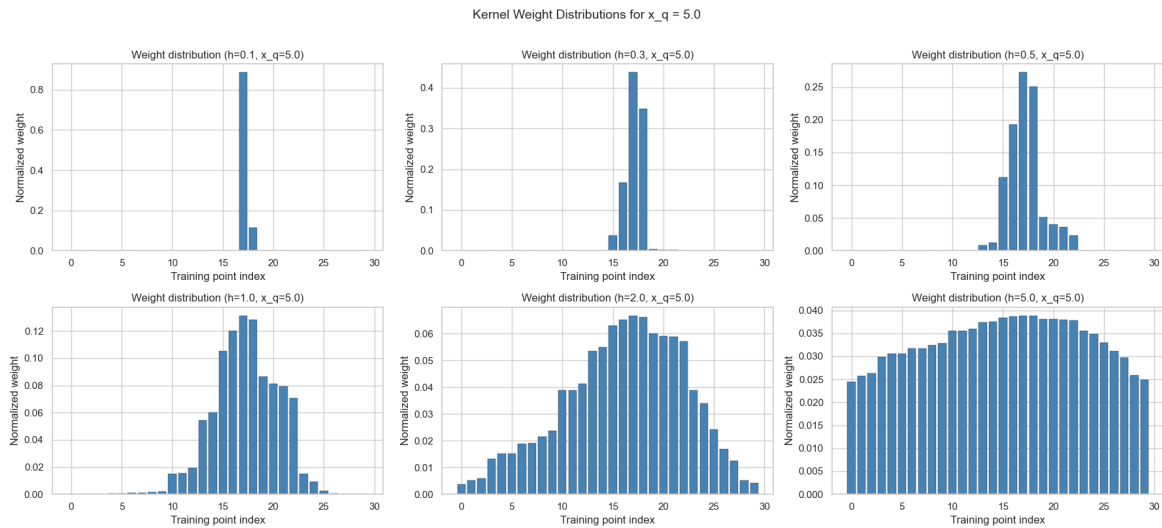


Figura 4: **Distribución de pesos $\tilde{w}_i$ para $x_q = 5.0$ con $h \in \{0.1, 0.3, 0.5, 1.0, 2.0, 5.0\}$** (mismo grid 2×3). Eje X = índice del punto de entrenamiento, Eje Y = peso normalizado (0 a 1). **Patrón clave:** h pequeño $\rightarrow$ spike sobre el vecino más cercano (toda la masa de probabilidad en un solo punto); h grande $\rightarrow$ distribución uniforme (todos los puntos con peso $\approx 1/30$). Con $h = 0.5$, $\sim 5-7$ puntos reciben pesos significativos, generando el suavizado óptimo.

Las Figuras 3 y 4 son dos caras de la misma moneda: la forma de la distribución de pesos *determina* directamente la calidad de la predicción. Un spike $\rightarrow$ overfitting; una distribución plana $\rightarrow$ underfitting; una campana estrecha $\rightarrow$ equilibrio óptimo.

Cuadro 2: Distribución cualitativa de pesos $\tilde{w}_i$ para $x_q = 5,0$

h	Forma de la distribución	Implicación
0,1	Spike único	Solo el vecino más cercano contribuye
0,3	Muy concentrada	2–4 puntos cercanos dominan
0,5	Concentrada	~5–7 puntos cercanos contribuyen
1,0	Moderadamente ancha	~10 puntos contribuyen significativamente
2,0	Ancha	La mayoría de puntos contribuyen
5,0	Casi plana	Los 30 puntos tienen pesos $\approx 1/30$

3.4.5. La curva MSE en forma de U: sesgo-varianza

La curva de MSE en función de h tiene la forma clásica de U :

$$\underbrace{\text{MSE}(h)}_{\text{error total}} = \underbrace{\text{Sesgo}^2(h)}_{\substack{\uparrow \text{ con } h \\ \text{(infraajuste)}}} + \underbrace{\text{Varianza}(h)}_{\substack{\downarrow \text{ con } h \\ \text{(sobreaajuste)}}$$

y el óptimo se encuentra en el mínimo de esa suma. En este dataset, el mínimo se alcanza en $h \approx 0,3$.

Por que la curva de MSE tiene forma de U?

Este es el *dilema sesgo-varianza* clásico del aprendizaje automático:

- **Para h pequeño:** poca bias (el modelo puede ajustarse a cualquier forma local) pero mucha varianza (pequeños cambios en los datos de entrenamiento cambian mucho la predicción). El ruido domina.
- **Para h grande:** mucha bias (el modelo ignora la estructura local) pero poca varianza (la media global es estable). La señal se pierde.
- **El óptimo** balancea ambas componentes. En la práctica, se encuentra mediante validación cruzada leave-one-out (LOO-CV), que para estimadores de kernel tiene una fórmula cerrada eficiente.

4. Ejercicio 2: AAKR para Detección de Fallos en Sensores

4.1. Descripción del dataset multivariado

El segundo ejercicio amplía AAKR al caso bidimensional (dos sensores correlacionados) para simular un sistema de monitorización industrial:

```

1 np.random.seed(42)
2 n_memory = 200
3
4 # Sensor 1: condicion de operacion
5 S1_memory = np.random.uniform(0, 10, n_memory)
6
7 # Sensor 2: respuesta del sistema (funcion de S1 + ruido)
8 S2_memory = (0.5 * np.sin(S1_memory)
9             + 0.3 * S1_memory
10            + np.random.normal(0, 0.1, n_memory))
11
12 X_memory = np.column_stack([S1_memory, S2_memory])

```

Listing 4: Generación del dataset de 2 sensores correlacionados

La función que relaciona ambos sensores en condiciones normales es:

$$S_2 = 0,5 \cdot \sin(S_1) + 0,3 \cdot S_1 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0,1^2). \quad (10)$$

Esta relación combina un componente oscilatorio ($0,5 \sin(S_1)$) con una tendencia lineal creciente ($0,3 S_1$), produciendo una curva no lineal monótonamente creciente con ondulaciones.

Conjunto de prueba con fallos

```

1 n_test_normal = 100
2 n_test_fault = 50      # 25 drift + 25 bias
3
4 S1_test = np.random.uniform(0, 10, n_test_normal +
5                             n_test_fault)
6 S2_base = (0.5 * np.sin(S1_test) + 0.3 * S1_test
7           + np.random.normal(0, 0.1, n_test_normal +
8                               n_test_fault))
9
10 # Insercion de fallos
11 S2_fault = S2_base.copy()
12 S2_fault[n_test_normal : n_test_normal + 25] += 1.0    #
13     drift +1.0
14 S2_fault[n_test_normal + 25 :] += 2.0    # bias
15     +2.0
16
17 X_test = np.column_stack([S1_test, S2_fault])
18 labels = np.array([0]*n_test_normal + [1]*n_test_fault)

```

Listing 5: Conjunto de prueba con fallos de drift y bias

El conjunto de test contiene 150 observaciones: 100 normales y 50 falladas. Los fallos se introducen exclusivamente en el Sensor 2 como desplazamientos aditivos:

- **Drift** (+1,0 unidades en S_2): simula una deriva gradual del sensor, como la que puede ocurrir por depósitos o desgaste del transductor.

- **Bias** (+2,0 unidades en S_2): simula un *offset* constante, típico de un fallo de calibración o de un cortocircuito parcial.

4.1.1. Visualización del dataset 2D

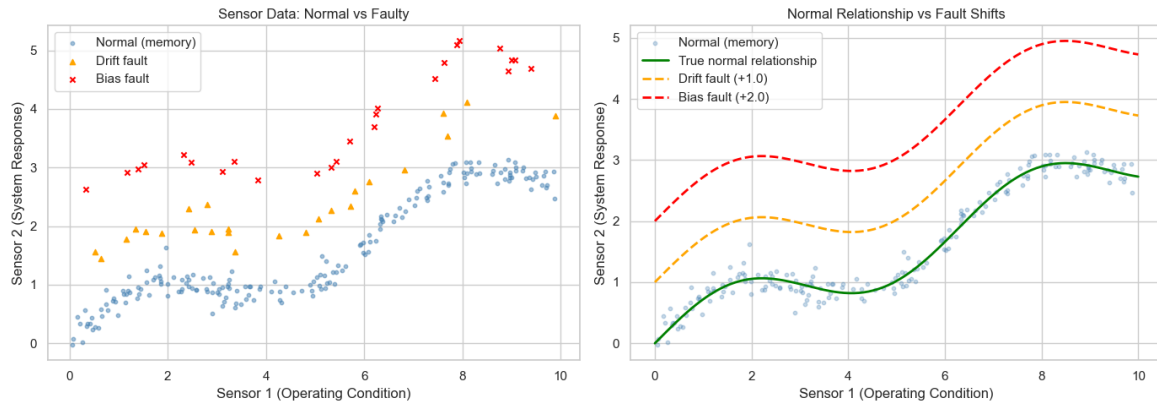


Figura 5: **Dataset 2D de sensores correlacionados**. Panel izquierdo: espacio (S_1, S_2) con los datos de memoria (puntos azules), los fallos de drift (+1 unidad en S_2 , triángulos naranjas) y los fallos de bias (+2 unidades en S_2 , cruces rojas). Panel derecho: relación normal $S_2(S_1)$ (curva verde) vs. las curvas de fallo (naranja y roja, desplazadas verticalmente). Observar que los fallos se alejan sistemáticamente de la variedad normal: AAKR los detectará porque no puede reconstruirlos con precisión a partir de la memoria normal.

La clave de la Figura 5 es que los puntos fallados (naranja y rojo) están *fuera* de la nube azul normal. AAKR proyectará cada punto fallado al punto de la variedad normal más cercano, generando un residuo proporcional al desplazamiento $\Delta \in \{1.0, 2.0\}$.

Por que normalizar con `cdist` en lugar de calcular distancias manualmente?

- En el caso multidimensional ($d \geq 2$), la distancia euclidiana en bruto puede estar dominada por la *escala* de las features: si $S_1 \in [0, 10]$ y $S_2 \in [0, 100]$, la componente de S_2 dominaría.
- `cdist` de `scipy.spatial.distance` calcula distancias entre *todos los pares* de puntos de forma vectorizada, aprovechando operaciones BLAS optimizadas $\rightarrow$ entre $10\times$ y $100\times$ más rápido que bucles Python.
- Si las escalas de los sensores son muy diferentes, es recomendable normalizar antes de calcular distancias: dividir cada sensor por su desviación estándar en la memoria normal.
- En este TD los dos sensores tienen escalas similares ($S_1 \in [0, 10]$, $S_2 \in [0, 4]$ aproximadamente), por lo que no es necesario normalizar.

4.2. Implementación multidimensional

La extensión de AAKR a múltiples dimensiones es directa: las distancias escalares se sustituyen por distancias euclidianas d -dimensionales. Se utiliza la función `cdist` de `scipy.spatial.distance` para un cálculo eficiente:

```

1 from scipy.spatial.distance import cdist
2
3 def aakr_reconstruct(X_memory, X_query, h):
4     """
5     AAKR multidimensional.
6
7     Parametros
8     -----
9     X_memory : ndarray (n, d) -- observaciones normales (
10         memoria)
11     X_query  : ndarray (m, d) -- observaciones a reconstruir
12     h        : float          -- ancho de banda del kernel
13
14     Retorna
15     -----
16     X_reconstructed : ndarray (m, d) -- reconstruccion
17         esperada
18     """
19     # Distancias euclidianas: forma (m, n)
20     distances = cdist(X_query, X_memory, metric="euclidean")
21
22     # Pesos del kernel gaussiano y normalizacion
23     weights = np.exp(-distances**2 / (2 * h**2))
24     weights = weights / weights.sum(axis=1, keepdims=True)
25
26     # Reconstruccion: media ponderada de la memoria
27     X_reconstructed = weights @ X_memory # (m,n)x(n,d)->(m
28         ,d)
29
30     return X_reconstructed

```

Listing 6: AAKR multidimensional (auto-asociativo)

Modo auto-asociativo: en AAKR multidimensional, tanto la entrada como la salida son el mismo vector $\mathbf{x}$. La memoria almacena observaciones completas $(\mathbf{x}_i, \mathbf{x}_i)$. Al consultar $\mathbf{x}_q$, el modelo devuelve $\hat{\mathbf{x}}_q$: el punto de la variedad normal más cercano a $\mathbf{x}_q$ según la ponderación gaussiana.

4.3. Pregunta 1: Análisis de la reconstrucción

4.3.1. Reconstrucción de datos normales y fallados

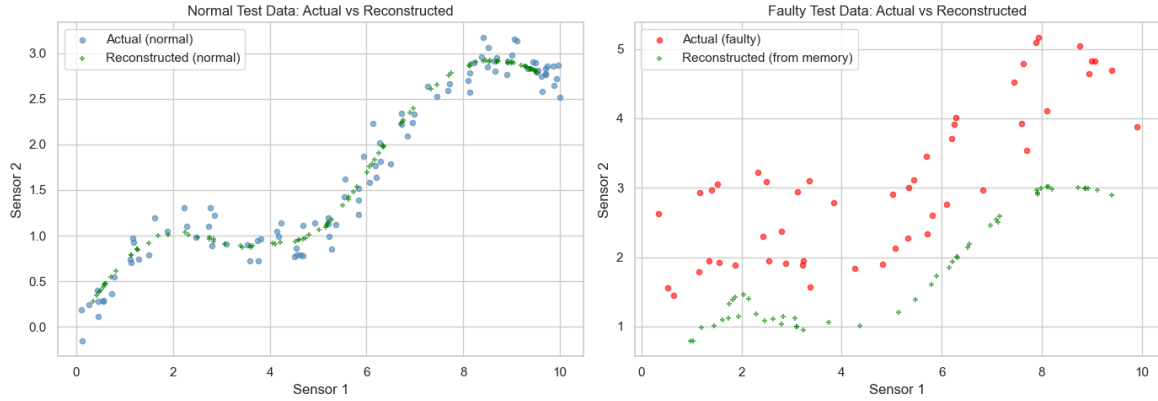


Figura 6: **Reconstrucción AAKR para datos normales y fallados.** Panel izquierdo: datos normales actuales (azul) vs. reconstruidos por AAKR (cruces verdes). La reconstrucción es casi perfecta ($MSE \approx 0.01$): los puntos azules y las cruces verdes se superponen casi completamente. Panel derecho: datos fallados actuales (rojo) vs. reconstruidos (cruces verdes). Las cruces verdes quedan en la variedad normal, mientras que los puntos rojos están desplazados: el gap entre ellos es la señal de fallo. $MSE \approx 1.6$.

Por que la reconstruccion falla en datos fallados?

La memoria *solo contiene* comportamiento normal. Para toda nueva observación $\mathbf{x}_q$, AAKR busca los vecinos más similares en esa memoria normal:

- Si $\mathbf{x}_q$ es normal, los vecinos más cercanos son puntos normales similares $\rightarrow$ reconstrucción $\hat{\mathbf{x}}_q \approx \mathbf{x}_q \rightarrow$ residuo pequeño.
- Si $\mathbf{x}_q$ es fallado (desplazado +1 o +2 unidades en S_2), el vecino más cercano en la memoria es el punto normal con el mismo S_1 pero S_2 sin el shift. Por eso la reconstrucción $\hat{x}_{q,S_2} \approx$ valor normal $\rightarrow$ residuo $r_q \approx$ shift de fallo.
- Este mecanismo es por diseño: AAKR no puede “alucinar” observaciones falladas porque no las tiene en memoria.

Para los 100 puntos de test normales:

$$MSE_{S_2}^{(\text{norm})} \approx 0,010.$$

Para los 50 puntos fallados:

$$MSE_{S_2}^{(\text{fault})} \approx 1,6,$$

dos órdenes de magnitud superior al caso normal. El residuo recoge directamente el desplazamiento:

$$r_q \approx \|\Delta\| \in [1,0, 2,0].$$

4.4. Pregunta 2: Sistema de detección basado en residuos

4.4.1. Cálculo de residuos y umbral

```

1 # Reconstrucción de todos los puntos de test
2 X_rec = aakr_reconstruct(X_memory, X_test, h=0.5)
3
4 # Residuo euclidiano por observacion
5 residuals = np.linalg.norm(X_test - X_rec, axis=1)
6
7 # Separacion normal / fallo
8 res_normal = residuals[:n_test_normal]
9 res_fault = residuals[n_test_normal:]
10
11 # Umbral 3-sigma sobre residuos normales
12 threshold = res_normal.mean() + 3 * res_normal.std()
13 print(f"Umbral (3-sigma): {threshold:.4f}")
14
15 # Clasificacion: 0=normal, 1=fallo
16 y_pred = (residuals > threshold).astype(int)

```

Listing 7: Cálculo de residuos y umbral de detección 3-sigma

El umbral calculado mediante la regla de los 3 sigmas es:

$$\tau = \mu_r^{(\text{norm})} + 3\sigma_r^{(\text{norm})} \approx 0,50.$$

4.4.2. Distribución de residuos e histograma

La figura siguiente muestra la separación entre distribuciones de residuos normales y fallados, junto con la matriz de confusión resultante:

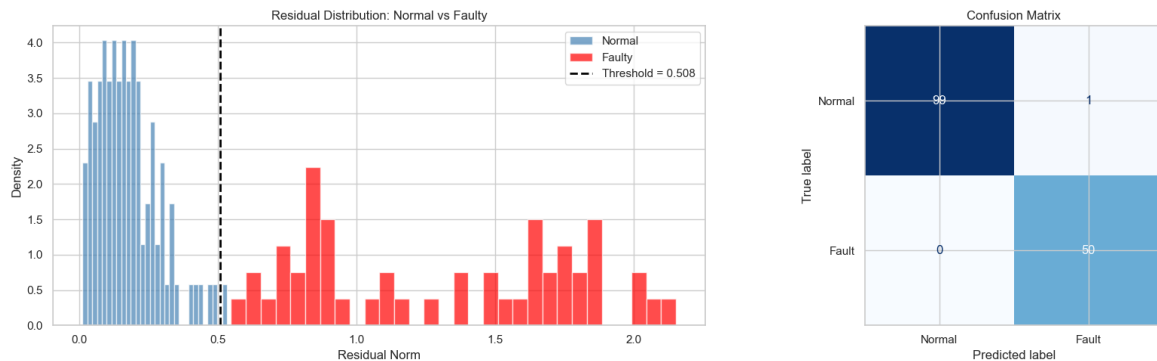


Figura 7: **Detección basada en residuos.** Panel izquierdo: histograma solapado de residuos normales (azul) y fallados (rojo), con línea negra discontinua indicando el umbral $\tau \approx 0.50$ (regla 3σ). Observar la escasísima superposición entre ambas distribuciones: los residuos normales se concentran en $[0, 0.3]$ y los fallados en $[0.8, 2.2]$. Panel derecho: matriz de confusión coloreada (verde = clasificaciones correctas, rojo = errores). Accuracy = 99.33 %.

El histograma de la Figura 7 es la mejor evidencia visual de por qué la detección es casi perfecta: los residuos normales y fallados forman distribuciones *bien separadas*. El umbral $\tau \approx 0.50$ cae en el *gap* entre ellas, garantizando que casi todos los fallos superen el umbral y casi ninguna observación normal lo supere.

4.4.3. Clasificación en el espacio de sensores

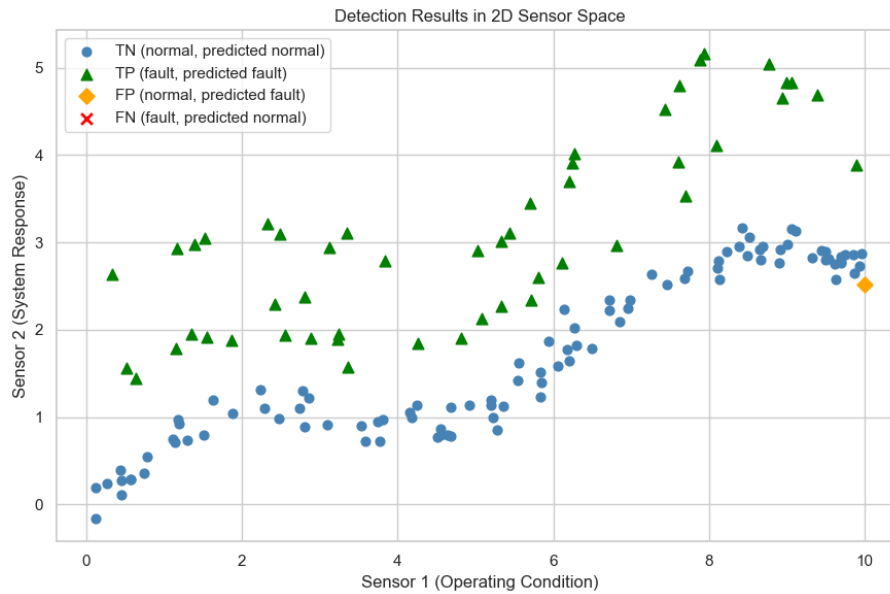


Figura 8: **Espacio 2D de sensores con clasificación de los 150 puntos de test.** Cuatro categorías: TP (triángulos verdes) = fallos correctamente detectados; TN (círculos azules) = normales correctamente clasificados; FP (rombos naranjas) = falsas alarmas (normales clasificados como fallo); FN (cruces rojas) = fallos no detectados. Observar: 1 FP (un punto normal atípico en la frontera de la nube normal) y 0 FN (todos los fallos detectados).

La Figura 8 revela dónde ocurre el único error de clasificación: el FP es un punto normal que por azar se encuentra en el extremo de la distribución normal, generando un residuo ligeramente superior al umbral. No hay ningún FN porque los puntos fallados están desplazados lo suficiente ($\geq +1$ unidad) para superar cómodamente el umbral de 0.50.

4.4.4. Resultados de clasificación

Cuadro 3: Matriz de confusión — Detector AAKR ($h = 0,5$, $\tau \approx 0,50$)

Real \ Predicción	Pred. NORMAL	Pred. FALLO	Total
Real NORMAL	99	1	100
Real FALLO	0	50	50
Total predicho	99	51	150

Cuadro 4: Métricas de evaluación del detector

Métrica	Valor	Fórmula
Accuracy	0,9933	$(TP + TN) / N$
Precision	0,9804	$TP / (TP + FP)$
Recall	1,0000	$TP / (TP + FN)$
F1 Score	0,9901	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$
Falsos positivos (FP)	1	Normal clasificado como fallo
Falsos negativos (FN)	0	Fallo no detectado

4.4.5. Análisis de los resultados

Separación de distribuciones. Los residuos de los datos normales se concentran por debajo de $\approx 0,3$ (reconstrucción precisa), mientras que los residuos de los datos fallados se sitúan entre 0,8 y 2,2 (proporcionales al desplazamiento Δ). Las dos distribuciones tienen escasísimo solapamiento, lo que hace que la tarea de clasificación sea prácticamente perfecta con un umbral simple.

1 falso positivo, 0 falsos negativos. El único error es un punto normal que, por azar, presenta una combinación de ruido que lo aleja lo suficiente de la memoria para superar $\tau \approx 0,50$. La probabilidad teórica de este evento bajo la regla de los 3 sigmas es $\approx 0,3\%$, consistente con 1 FP en 100 puntos normales.

Los 0 falsos negativos son especialmente importantes en el mantenimiento industrial: un **fallo no detectado (FN)** puede conducir a una avería grave, parada no planificada o accidente. Un **falso positivo (FP)** solo desencadena una inspección innecesaria, con un coste mucho menor.

Compromiso precisión–exhaustividad. La elección del umbral τ determina directamente el punto de operación en la curva ROC. La regla de los 3 sigmas es conservadora (umbral alto): garantiza pocos FP, pero podría dejar escapar fallos más sutiles ($\Delta < 0,5$). Para detectarlos, habría que reducir τ , aceptando más FP a cambio de mayor exhaustividad (Recall).

5. Discusión Comparativa

5.1. AAKR frente a otros métodos de detección de anomalías

Cuadro 5: Comparativa de AAKR con métodos alternativos

Método	Requiere datos de fallo	Escalabilidad	Interpretabilidad
AAKR	No	Baja	Alta
PCA / DPCA	No	Alta	Media
Isolation Forest	No	Alta	Baja
Autoencoder	No	Alta	Baja
SVM (One-Class)	No	Media	Media
Random Forest	Sí	Alta	Media
LSTM / TCN	Sí (ideal)	Alta	Baja

5.2. Ventajas de AAKR

1. **Aprendizaje no supervisado de anomalías.** Solo requiere datos normales, que son abundantes en sistemas industriales. Los datos de fallo son escasos, costosos de obtener y difíciles de etiquetar.
2. **Sin fase de entrenamiento.** La memoria se construye en tiempo lineal $\mathcal{O}(n)$ y puede actualizarse en tiempo real simplemente añadiendo o eliminando observaciones.
3. **Modela correlaciones entre sensores.** Al operar sobre vectores $\mathbf{x} \in \mathbb{R}^d$, AAKR captura la relación entre todos los sensores simultáneamente. Un fallo que afecta solo a un sensor rompe esta correlación y genera un residuo grande.
4. **Interpretabilidad del residuo.** El residuo r_q tiene unidades físicas (las mismas que los sensores) y puede descomponerse por componentes para identificar *qué* sensor está fallando y *cuánto* se ha desviado.
5. **Umbral adaptable.** La regla de los 3 sigmas proporciona un punto de partida robusto, pero el umbral puede ajustarse según la tolerancia al riesgo de la aplicación.

5.3. Limitaciones de AAKR

1. **Escalabilidad.** El coste de predicción es $\mathcal{O}(nm)$ para m consultas y n puntos de memoria. Para n grande (millones de observaciones), esto puede ser prohibitivo. Soluciones: submuestreo de la memoria, estructuras k -d tree, o aproximaciones de kernel.

2. **Sensibilidad a h .** La elección del ancho de banda requiere validación cruzada o conocimiento experto. Un h mal calibrado genera muchos FP (h pequeño) o no detecta fallos pequeños (h grande).
3. **Datos normales heterogéneos.** Si el comportamiento normal varía significativamente (e.g., distintos regímenes de operación), AAKR puede generar residuos grandes en zonas poco pobladas de la memoria, incluso para observaciones normales.
4. **Maldición de la dimensionalidad.** La distancia euclidiana pierde significado cuando d es grande. En espacios de alta dimensión conviene normalizar las variables o aplicar reducción de dimensionalidad previa.
5. **Fallos correlacionados sutiles.** Si el fallo desplaza múltiples sensores de forma consistente con la variedad normal, el residuo puede ser pequeño y el fallo no se detecta.

5.4. Conexión con el estimador de Nadaraya-Watson

AAKR pertenece a la familia de los *estimadores de Nadaraya-Watson*, que son la base de la regresión no paramétrica por kernel. Matemáticamente, la predicción de AAKR coincide exactamente con este estimador cuando se usa un kernel gaussiano isotrópico. Por tanto, AAKR hereda las propiedades teóricas bien establecidas de ese estimador:

- *Consistencia:* convergencia en media cuadrática a la función de regresión verdadera cuando $n \rightarrow \infty$, $h \rightarrow 0$ y $nh \rightarrow \infty$.
- *Tasa óptima:* $\mathcal{O}(n^{-4/(d+4)})$ bajo condiciones de suavidad estándar.

5.5. Aplicaciones industriales reales

AAKR y sus variantes se han desplegado con éxito en los siguientes dominios industriales:

- **Energía nuclear:** monitorización de temperaturas y presiones en el circuito primario de reactores (aplicación original de Mott et al.).
- **Turbinas eólicas:** detección de fallos en cojinetes, generadores y palas a partir de señales de vibración y temperatura.
- **Aviación:** monitorización de salud de motores de avión (Engine Health Management), donde los fallos deben detectarse con mínimos FN.
- **Ferrocarril:** detección de anomalías en sistemas de frenado y rodadura.
- **Procesos químicos:** monitorización de reactores, columnas de destilación y compresores.

En todos estos casos, la propiedad clave es la misma: el comportamiento normal está bien caracterizado (muchas horas de operación sin fallos) y los datos de fallo son escasos o no disponibles en el momento del diseño del sistema.

6. Conclusiones

Este TD ha demostrado que AAKR es un algoritmo simple, elegante y efectivo para la detección de fallos en sensores industriales. Las principales conclusiones son:

- 1. Implementación directa y vectorizable.** Los tres pasos del algoritmo (distancias, kernel, media ponderada) se implementan en menos de 10 líneas de NumPy con vectorización completa. La extensión al caso multidimensional no requiere cambios conceptuales: basta sustituir las distancias escalares por `cdist` euclidiana.
- 2. Calidad de predicción competitiva.** Con h bien elegido, AAKR alcanza un MSE del orden del nivel de ruido ($\approx 0,02$ para $\sigma = 0,15$), recuperando fielmente la función $\sin(x)$ subyacente sin especificar ninguna forma funcional *a priori*.
- 3. El ancho de banda h es el único hiperparámetro crítico.** La curva MSE en función de h tiene forma de U: valores pequeños producen sobreajuste (solo el vecino más cercano) y valores grandes producen infraajuste (media global). El óptimo en el dataset 1-D analizado es $h \approx 0,3$, identificado por barrido sistemático.
- 4. Visualización de pesos como herramienta de diagnóstico.** La distribución de pesos $\tilde{w}_i$ para un punto de consulta dado revela directamente si el modelo está en régimen de overfitting (spike), underfitting (uniforme) o ajuste óptimo (campana estrecha).
- 5. Detección de fallos casi perfecta.** En el escenario bidimensional con fallos de drift $(+1,0)$ y bias $(+2,0)$, AAKR alcanza: Accuracy = 99,33 %, Recall = 100 % (0 fallos no detectados) y F1 = 99,01 %. El único error es 1 falso positivo, irrelevante en mantenimiento industrial.
- 6. El mecanismo de detección es intuitivo y físicamente interpretable.** La memoria normal permite reconstruir con alta precisión observaciones normales (residuo pequeño), pero no puede reconstruir observaciones falladas porque están fuera de la variedad normal (residuo grande). La regla de los 3 sigmas proporciona un umbral robusto e interpretable con tasa de falsas alarmas teórica del 0,3 %.
- 7. Aplicabilidad y limitaciones industriales.** AAKR es idóneo para sistemas con comportamiento normal bien caracterizado y datos de fallo escasos o no disponibles. Su principal limitación es la escalabilidad: para memorias de millones de observaciones se requieren aproximaciones (submuestreo, k -d trees). En sistemas con alta dimensionalidad o múltiples regímenes de operación, se recomienda complementarlo con reducción de dimensionalidad o agrupamiento previo.